Bouncy Golf 2D Technical Document

- **Mohamed Ibrahim**

Overview

Game Inspiration

- Inspired by the arcade game "Wall Breaker."

Objective

- The objective is to launch the ball toward the goal by aiming accurately.

Features

- The game includes three levels and a simple user interface (UI) to navigate between them.

Credits

Assets

1. **Background**: OpenGameArt.org
2. **Ball**: Pinterest
3. **Background Music (BGM)**: GMTK 2024 Open Assets
4. **Hover Sound Effects (SFX)**: Unity Asset Store
5. **OnClick SFX**: Unity Asset Store

Controls

Input Methods

- **Mouse**:
    - Left-click to shoot.
    - Hold left-click to aim the projectile.
- **Keyboard**:
    - Press "V" to enable debug view of colliders.
    - Use WASD keys to move the camera when debug mode is enabled.

Game Features

- Dynamic physics-based gameplay inspired by arcade mechanics.
- Custom-built Entity Component System (**ECS**) architecture.
- Debug mode for viewing colliders and physics interactions.
- Projectile trails implemented using the Object Pooling technique.
- Players must shoot the ball toward the goal while predicting its trajectory.
- The ball passes through obstacles of the same color.
- The ball deflects upon hitting obstacles of a different color.

System Architecture

Entity Component System (ECS)

1. **World.h**
    - Responsible for creating managers and entities.
2. **Entity.h**
    - Contains all entity data, including a list of attached components.
    - Components are stored in a map using their eComponentType as the key.
3. **IComponent.h**
    - Defines an interface for components with virtual methods: start, update, clean, and clone.
    - New components can inherit and implement their own functionality.
4. **ISystem.h**
    - Provides an interface to create systems based on component types.
5. **SystemManager.h**
    - Maintains a map of systems and updates entities' behavior for each system.
6. **System Types**
    - Includes:
        - **Render System**
        - **Custom Scripts**
        - **Physics System**
        - **Movement System**

- ❖ **Particle System**

7. **Event Subscription**

    - Every system subscribes to events (OnEntityAdded and OnEntityRemoved) in the Start() method.
    - Example: RenderSystem listens for new entities with RenderComponent, while PhysicsSystem checks for Rigidbody and Collider components.

Render System

- **Structure**
    - Maintains two lists of entities with RenderComponent:
        1. One for rendering sprites.
        2. Another for updating UI elements.
- **Update Method**
    - Sorts components by their renderOrder() defined in RenderComponent.h in a list.
    - Updates the component's behavior, e.g., SpriteRenderer.h would update animations if any.
- **Render Method**
    - Renders the UI list first and then renders the sorted sprites.

Physics System

- **Structure**
    - Contains a map that holds EntityID and PhysicsEntity (a struct storing information such as Collider, Rigidbody, and the associated entity).
- **Start Method**
    - Populates the map with PhysicsEntity objects for entities that have both Rigidbody and Collider components.
- **Update Method**
    - Loops through the map of physics objects every frame.
    - Performs physics calculations at a fixed time step (60 FPS).
    - Updates entity positions based on velocity.

    - Resolves collisions and triggers events:
        1. CollisionEnter, CollisionStay, and CollisionExit.
        2. If the collider is set to trigger, invokes TriggerEnter, TriggerStay, and TriggerExit.
    - Collision events are inspired by Unity's collision system.
- **Render Method**
    - Renders the bounds of collider shapes for debugging purposes.
    - Debug mode can be toggled by pressing the "V" key.
- **RemoveEntity Method**
    - Removes Rigidbody and Collider components from the system's map when an entity is deleted.

Custom Script System

- **Inspiration**
    - Inspired by Unity's MonoBehaviour, allowing for custom scripts to be created.
- **Structure**
    - Contains a list of pairs with entities and their BaseScriptComponent.
- **Start Method**
    - Populates the list based on objects with attached BaseScriptComponent.
- **Update Method**
    - Loops through the list of pairs and updates each component every frame.

Movement System

- **Structure**
    - Handles world origin to simulate camera-based movement.
- **Update Method**
    - Loops through each entity's transform component and adjusts its position relative to the camera's position to simulate camera movement.
    - WASD keys can be used to move the camera in debug mode.

Other Interesting Elements

- **Object Pooling**
    - Implemented using the ObjectPool.h template class.
    - Used for rendering projectile trails (see EntityPool.h).
- **Factory**
    - GameObjectFactory.h is used for creating game objects.
- **Event System**
    - CEvent.h manages event callbacks.
    - **Structure**:
        1. Contains a list of callbacks based on types.
        2. Subscribe: Adds a callback to the list.
        3. Invoke: Loops through the callbacks, validates, and calls the functions.
        4. Clear: Removes all callbacks in the list.
- **Input Manager**
    - InputManager.h handles all keyboard and mouse inputs.
    - **Structure**:
        1. Contains a map of booleans with integers as keys.
        2. getKeyDown: Returns true when a key is pressed once.
        3. getKey: Returns true while a key is held down.
        4. getKeyUp: Returns true when a pressed key is released.
- **Timer**
    - Timer.h is a singleton class responsible for storing delta time for the entire world.
- **Utilities**
    - Utils.h contains static utility classes for debugging, randomization, and math functions.
- **Physics Utilities**
    - PhysicsUtils.h handles collision checks for types such as BoxVsBox and CircleVsCircle.
    - Implements raycasting behavior similar to Unity.

Overall Experience

I thoroughly enjoyed the learning experience and the overall journey of creating this project. I want to especially thank Ubisoft Toronto for allowing me to participate in the **Ubisoft NEXT** programming challenge.

This journey has boosted my confidence in working with **C and C++** languages. Initially, structuring classes and handling behaviors was challenging. However, after deciding to adopt an ECS architecture, the process became more manageable and enjoyable. I am now confident in building projects using any API, and I believe this experience has provided me with the foundation to create 2D games with my custom API.